

博客讲稿与面试追问题库 (A 版)

> 配套博客: 01_gazebo_slam_navigation.md、
02_m1_slam_principle.md

>

> 用法:

- > 1. 先读"总纲", 直到能不看稿子讲出 30 秒版和 3 分钟版;
- > 2. 逐节读"大白话 + 类比", **每节合上文档用嘴讲一遍**;
- > 3. 追问题库用来查漏——每道题先自己答, 再对照参考答案;
- > 4. 最后做 B 计划: 把每节讲给我听, 我帮你挑毛病。

总纲

30 秒版 (复试第一问"你做过什么")

> "我做了个无人系统仿真项目, 主线是'先车后船'。先用 TurtleBot3 小车在 Gazebo 里跑通了完整闭环: 激光雷达建图、保存地图、再加载地图导航到目标点。为了搞懂 SLAM 原理, 我还用 Python 手写了一个简化版 SLAM——占位栅格地图加扫描匹配——并且把它和 ROS 里标准的 gmapping 做了同一份数据的对比实验。"

3 分钟版 (展开讲)

> "项目分四层技术: 感知、规划、控制、协同。算法跟载体无关, 所以先在地面小车上验证, 后面迁移到船。"

>

> 第一件事是跑通官方流程: Gazebo 里开一辆带激光雷达的小车, 用 gmapping 实时建图, 边遥控边看 RViz 里的栅格地图长出来; 地图存下来之后, 再加载地图启动 AMCL 定位 + move_base 导航, 在 RViz 里点一个目标点, 小车自己规划路径走过去。这一步我踩了不少坑: 仿真里车松键不停、地图文件路径没跟着搬导致导航起不来、RViz 里车的位置跟 Gazebo 对不上——最后发现是初始位姿没设在建图起点。

>

> 跑通之后我觉得 gmapping 是黑盒, 就手写了一个最小闭环 SLAM: 用 log-odds 更新栅格地图, 用相关扫描匹配修正里程计的漂移, 再加一个验收门槛防止误匹配。结果轨迹误差从 0.496 米降到 0.315 米, 地图和真值的 IoU 从 0.126 升到 0.346。

>

> 最后我把同一份仿真数据同时喂给手写版和 gmapping 做对比。有意思的是在空房间这种全对称环境里, gmapping 的粒子滤波出现感知混叠, 地图画糊了; 我的手写版用穷举扫描匹配反而稳。这说明算法失效模式跟环境强相关, 也让我理解了为什么真实系统要有特征、回环检测这些手段。"

博客 01: Gazebo 建图与导航

1. 背景 (先车后船)

****大白话****: 无人系统不管天上飞、地上跑还是水里游, 核心技术都是同一套: 先感知环境、再规划路径、然后控制执行, 多台设备还要协同。我先把这套技术在小车上验证, 之后把同样的算法换到船上。

****类比****: 考驾照先拿小车练手, 练熟了换大车原理一样, 只是车感不同。

****追问****:

- Q: 为什么先做小车再做船?

A: 算法跟载体无关, 变的只是模型、动力学、传感器。小车仿真环境成熟、资料多、成本低, 先跑通闭环, 再换船的模型和传感器 (比如激光换声呐)。

- Q: 四层技术具体指什么?

A: 感知 (SLAM 建图/定位)、规划 (路径规划)、控制 (跟踪路径)、协同 (多机编队)。

2. 环境 (ROS1 vs ROS2)

****大白话****: ROS 是机器人操作系统, 负责让各个程序互相通信; Gazebo 是 3D 仿真器, 提供一个虚拟世界。我用的 ROS1 Noetic 是 20.04 上最成熟的版本, ROS2 留给后面集群阶段。

****关键概念****:

- 节点: 一个程序 (比如建图节点、遥控节点)
- 话题: 节点之间传数据的"频道" (比如 /scan 传激光数据、/cmd_vel 传速度指令)
- launch 文件: 一次启动一组节点的配置文件

****追问****:

- Q: ROS1 和 ROS2 有什么区别?

A: ROS1 有个中心节点 (roscore) 做注册, 多机通信要手动配; ROS2 用 DDS 分布式通信, 天然支持多机集群, 所以集群阶段用 ROS2。

- Q: 话题和服务的区别?

A: 话题是持续的发布/订阅, 一方发多方收; 服务是一问一答, 请求响应。连续数据流用话题, 偶尔要个结果用服务。

3. 核心概念速览 (必背)

SLAM / gmapping

****大白话****: SLAM 就是"边开车边画地图, 同时知道自己画到哪了"。gmapping 是 ROS 里一种成熟的 SLAM 算法, 用粒子滤波实现。

****类比****: 你在一个黑屋子里举着激光笔边转边画墙的位置, 每走一步还记着"我大概在哪儿"—但你走路会漂, 所以每画一笔都要用刚扫到的墙反过来纠正"我其实在哪儿"。

栅格地图 (occupancy grid)

****大白话****: 把空间切成很多小方格, 每格记"这里有没有东西" (占据/空闲/未知)。激光打到的格子标"有墙", 穿过的格子标"空的"。

AMCL (定位)

****大白话****: 给车一张已知地图, 用粒子滤波回答"我现在在哪"。撒几千个粒子 (猜测), 每个粒子对照激光看猜得对不对, 对的活下来、错的淘汰, 最后聚成一团 = 定位成功。

****类比****: 拿着地图进商场, 先随便猜几个位置, 然后对着周围的店铺招牌 (激光) 逐个排除, 最后锁定自己站在哪。

move_base (导航)

****大白话****: 拿到"我要去 X 点"这个目标后, 全局规划器先在地图上找一条路, 局部规划器边走边躲障碍, 最后输出速度指令 /cmd_vel 给底盘。

TF 坐标系链 (必考)

****大白话****: 机器人身上所有部件的位置都是相对关系, TF 就是这些相对关系的链条: 地图(map) → 里程计起点(odom) → 机器人底盘(base) → 激光(base_scan)。要回答"墙在激光的哪个方向", 就把这些关

系串起来算。

- ``map → odom``: 由 AMCL 发布 (定位认为车在地图哪)
- ``odom → base_footprint``: 由里程计/仿真发布 (轮子走了多少)
- ``base_footprint`` 往下: 由机器人模型 (URDF) 发布 (激光装在车头多少厘米)

****追问**:**

- Q: SLAM 和定位有什么区别?

A: SLAM 是"地图也没有、位置也不知道"两个一起解; AMCL 是地图已知, 只解位置。所以建图用 gmapping, 建完图导航用 AMCL。

- Q: 为什么要有 odom 和 map 两个坐标系?

A: odom 是连续可信的局部参考 (从轮子累计), 但会漂移; map 是全局参考 (地图本身)。AMCL 负责把两者对齐, 相当于不断校正 odom 的漂移。

4. 建图实操 (三个终端各干什么)

****大白话**:** 终端 1 开虚拟世界 (车出现在房间里); 终端 2 启动建图算法并打开 RViz 显示地图生长; 终端 3 用键盘遥控车走一圈, 把房间每个角落都扫到。

****关键**:** 保存地图用 ``save_map.launch``, 生成两个文件: ``map.pgm`` (图像) 和 ``map.yaml`` (说明文件, 包括分辨率、原点、图像路径)。

****追问**:**

- Q: 为什么建图要让车慢慢走、转圈?

A: 激光要扫到房间所有区域, 慢慢走、少急转, 扫描匹配才不容易跟丢, 地图才完整清晰。

5. 导航实操 (RViz 两步)

****大白话**:** 导航启动后, 先点 ``2D Pose Estimate`` 告诉 AMCL "车大概在这里、朝这个方向" (给粒子一个初始猜测); 再点 ``2D Nav Goal`` 告诉它"去那儿"。之后你会看到: 粒子云聚拢 (定位成功) → 绿色全局路径出现 → 小车沿路走、遇障绕开 → 到达自动停。

****追问**:**

- Q: 为什么导航前必须设初始位姿?

A: AMCL 的粒子要撒在地图里, 总得先有个大概位置, 否则粒子在 20 米地图里乱撒, 收敛不了。设了初始位姿它才知道往哪儿撒。

- Q: 粒子云发散是什么意思?

A: 说明定位不确定, 粒子分散。通常重新设一次初始位姿, 或者让车动一动让激光对上有特征的地方。

6. 踩坑记录 (面试亮点, 每个都要会讲)

坑 1: 松键车不停

****大白话**:** 仿真里的车轮只认"最新一条速度指令"。遥控程序只在按下键的瞬间发一条速度, 松键不补发"速度 0", 所以车就保持最后一条指令继续跑。真车固件有看门狗 (几百毫秒没新指令自动停), 仿真里没这个机制。

****追问**:**

- Q: 怎么让车停?

A: 按 s 或空格, 它会发一条零速度指令; 别直接 Ctrl+C 退出遥控, 否则最后一条非零指令还在话题上。

- Q: 这说明仿真和真车的什么差别?

A: 仿真是理想化的, 缺少真车的安全机制 (看门狗、阻力), 所以仿真跑通不等于真车直接能用。

坑 2: map_server 启动即崩

****大白话****: `map.yaml` 里写的图片路径是绝对路径。把地图文件搬了家却没改 yaml, 加载时按旧路径找不到图片, 进程直接退出。

****追问****:

- Q: 以后搬地图要注意什么?

A: `map.pgm` 和 `map.yaml` 必须一起搬, 且 yaml 里的 `image:` 路径必须指向 pgm 的实际位置。

坑 3: RViz 里车的位置和 Gazebo 对不上 (最重要的坑)

****大白话****: 车在 Gazebo 里生成在 $(-2, -0.5)$, 但导航默认认为车在地图原点 $(0, 0)$ 。RViz 里画的车不是"真实位置", 而是"AMCL 认为的位置"。初始位姿没设在建图起点, 激光轮廓就永远和地图墙对不上。

****追问****:

- Q: 怎么确认初始位姿设对了?

A: 看激光轮廓和地图墙线是否重合, 再看粒子云是否聚在车上。

- Q: RViz 里车的模型位置是谁决定的?

A: 是 TF 链决定的: map→odom (AMCL 发布) →odom→base (里程计) →URDF。所以 AMCL 认为在哪, 车就画在哪。

坑 4: 不设初始位姿导航起不来

****大白话****: AMCL 没收到初始位姿就不发布 map→odom, 整个导航栈等不到 map 坐标系, 全链路卡死。这就是为什么"2D Pose Estimate 不能省"。

7. 验证清单 (会看就行)

****大白话****: 怎么知道系统真的好了: 激光有数据、TF 链路完整、粒子云收敛、激光和地图重合、发目标后出现路径且车到达。

8. 与手写 SLAM 的联系

****大白话****: gmapping 是"工业版", 我的 m1 是"原理版"—同样用栅格地图 + 扫描匹配, gmapping 多了粒子滤波和一堆工程细节。先看懂原理版, 再用 gmapping 就不晕了。

博客 02: 手写 2D 栅格 SLAM

1. 为什么先手写一遍

****大白话****: gmapping 一条命令就能建图, 但它是黑盒, 出了问题不知道为啥。手写一遍能回答三个问题: 为什么只用里程计会重影? 扫描匹配在修什么? 为什么地图越建越准?

****关键认知 (必背)****: SLAM 的核心是**地图和位姿互相修正**——位姿准了地图干净, 地图干净了位姿更准, 形成闭环。

****追问****:

- Q: 只用里程计为什么建图会重影?

A: 里程计有漂移 (距离放大、陀螺偏置、噪声), 误差一路累积, 绕一圈回来, 墙的位置对不上, 地图就拖花了。

2. 最小闭环的三个零件

2.1 占位栅格地图 + log-odds (必考)

****大白话****: 每个格子存一个"信心值" (log-odds)。激光穿过的格子, 信心减一点 (``lo_miss=-0.2``, 表示"这格是空的"); 激光打到的格子, 信心加一点 (``lo_hit=1.5``, 表示"这格有墙")。多帧叠加就是贝叶斯更新。

****为什么用 log-odds 而不是直接概率****:

1. 概率相乘 (贝叶斯) 在 log 域变成相加, 实现简单;
2. 多帧累加不会溢出 (概率会压到 0 或 1);
3. 加减对称, 地图可以"再相信"也可以"改主意"。

****两个保护细节****:

- 只把"激光到达之前"的区域标空闲, 给墙留 0.4m 余量, 防止斜射把薄墙擦掉;
- 已经是强占据的格子不再接受"空闲"更新, 保护已建好的墙。

****追问****:

- Q: lo_miss 为什么比 lo_hit 小?
A: 一束激光穿过的格子很多, 撞墙的格子只有一个, 如果不平衡, 空闲更新会淹没占据更新。
- Q: log-odds 和概率怎么换算?
A: ``l = log(p/(1-p))``, 反过来 ``p = 1/(1+exp(-l))``。

2.2 相关扫描匹配 (必考)

****大白话****: 里程计猜了个位置 (不一定准)。扫描匹配就在这个猜测附近试很多个位置, 把激光端点投影到地图上, 算"落在墙上的分数", 分数最高的位置就是更可信的位置。

****实现****: 两级搜索——粗搜 (0.2m / 2°) 先定位大概, 精搜 (0.05m / 0.5°) 再精确。地图先做 3×3 模糊, 让墙"变厚", 分数变化平滑, 不容易陷进错误的局部最优。

****追问****:

- Q: 匹配分数怎么算?
A: 把当前帧激光的所有端点, 按候选位姿投影到地图, 取它们落在的格子, 把占据概率加起来。
- Q: 为什么先粗搜再精搜?
A: 直接精搜太慢。粗搜步长大、覆盖范围大, 先锁定一个大概区域, 再在小范围里细搜, 速度快且不易漏。
- Q: 地图模糊是干什么的?
A: 让分数在墙附近连续变化, 形成一个平滑的"分数地形", 搜索时能顺着梯度找到最高点, 而不是陷进孤立的小尖峰 (局部最优)。

2.3 验收门槛

****大白话****: 不是每次匹配结果都采用。只有匹配分数明显优于里程计猜测时才接受, 否则退回猜测。防止地图还没建好时被噪声带偏。

3. 实验一: 基线 (数字要记住)

指标	只用里程计	扫描匹配修正
---	---	---
轨迹 RMSE	0.496 m	0.315 m
地图与真值 IoU	0.126	0.346

****大白话****: RMSE 是"估计轨迹和真实轨迹平均差多少"; IoU 是"地图里画的墙和真实墙重叠了多

少" (0~1, 越大越好)。加了扫描匹配, 轨迹误差降了 36%, 地图质量翻了一倍多。

****追问**:**

- Q: RMSE 和 IoU 分别说明什么?

A: RMSE 看位姿准不准, IoU 看地图像不像。两者都要看, 一个管"过程"一个管"结果"。

4. 实验二: 参数扫描 (重点讲三个结论)

****大白话**:** 同一份数据, 改两样东西——漂移大小和搜索窗口大小, 看精度和耗时怎么变。

三个结论 (必背):

1. ****漂移越大, 修正收益越明显**:** 距离误差大时, 修正把 RMSE 从 0.656 压到 0.374。
2. ****窗口太小会跟丢**:** 搜索范围不够, 找不到正确位置, 修正后误差反而变大 (0.453), IoU 最低。
3. ****窗口太大只是变慢**:** 精度没提升, 耗时从 54s 涨到 81s。

还有一个意外发现: 角偏置大那档, 轨迹 RMSE 修正后反而"更差", 但地图 IoU 修正后仍然更高 (0.331 vs 0.142)——因为那一档的里程计误差恰好和真值抵消了一部分。****单一指标会骗人, 要看多个指标**。**

****追问**:**

- Q: 搜索窗口大小由什么决定?

A: 取决于对里程计漂移的估计——漂移越大, 窗口就要越大; 但窗口大了慢, 所以要平衡。理想情况下窗口刚好盖住最大可能的漂移。

- Q: 为什么窗口太小会跟丢?

A: 真实位置在窗口外面, 搜索范围内找不到正确的匹配, 只能选一个"矮子里拔将军"的错误位置。

5. 实验三: 和 gmapping 同输入对比 (本篇最大亮点)

****大白话**:** 把同一份激光/里程计数据, 同时喂给手写版和 gmapping, 比谁的地图好。为了公平, 两张地图先对齐再算重叠度。

结果: 手写版地图与真值 IoU 0.346; gmapping 对齐后与手写版 IoU 只有 0.019——****gmapping 的地图是"涂抹"的**。**

****为什么** (必背):**

- 实验房间是 14m 空方形房间, 四面墙完全对称、内部零特征;
- gmapping 用粒子滤波, 这种环境出现****感知混叠 (perceptual aliasing)****: 某面墙的激光轮廓和对面墙几乎一样, 粒子"认错墙"后沿墙滑移, 绕一圈回来墙就对不上了, 画成宽约 26m 的涂抹带;
- 手写版用****穷举相关扫描匹配****: 每帧在局部窗口里找分数最高的位姿, 不依赖"粒子群猜得多样", 对称环境反而不容易滑移。

****追问**:**

- Q: 感知混叠是什么意思?

A: 不同位置的观测长得一样, 算法分不清"我在哪面墙旁边"。特征越少越对称, 越容易混叠。

- Q: 那 gmapping 是不是不如你的手写版?

A: 不能这么说。这是极端退化场景; 真实环境有箱子、门、柱子, 还有回环检测、多传感器, gmapping 是成熟工程实现。这个实验说明的是"算法失效模式跟环境强相关", 也说明单一算法要放在具体场景里评价。

- Q: 怎么保证对比公平?

A: 同一份 bag (完全相同的激光、里程计、时间戳), 跑完把两张地图按占据格互相关对齐再算 IoU。

同输入对比的三个坑 (讲了就是加分项)

1. 静态变换要发 `/tf_static`, 否则 gmapping 的 tf1 过滤器按扫描时间戳查询全部失败

(Dropped 100%) ;

2. gmapping 参数名是 `base\_frame/odom\_frame/map\_frame`，且低噪声数据要调小更新阈值、加大粒子数；
3. rosbag 回放同时戳跨话题投递顺序不定，扫描要延迟 0.2s 发布才能稳定。

6. 与真实系统的关系

****大白话****：我的手写版是"教学版"，真实系统是它的加强版：

- 相关扫描匹配 $\approx$ Cartographer 局部匹配 / 激光 ICP 的概念原型
- 两级搜索 $\approx$ 多分辨率搜索的简化版
- 地图模糊 + 验收门槛 $\approx$ 真实系统的鲁棒性处理
- gmapping 的粒子滤波 = 把"单点估计"升级成"一群粒子投票"，更稳但更重

****追问****：

- Q：下一步想做什么？

A：把 m1 包成 ROS 节点和 gmapping 在真实 Gazebo 环境（有特征）里对比；再了解回环检测，解决长距离漂移。

术语速查表（看到能认、能解释）

术语	一句话解释
SLAM	同时定位与建图
occupancy grid	栅格地图，格子标占据/空闲/未知
log-odds	概率的 log 形式，相加即贝叶斯更新
扫描匹配	在猜测位姿附近搜一个让激光最贴合地图的位置
感知混叠	不同位置的观测相似，位姿产生歧义
粒子滤波	用一群带权重的猜测（粒子）逼近真实状态分布
RBPF	gmapping 用的粒子滤波变体，每个粒子带轨迹+地图
AMCL	已知地图下的粒子滤波定位
move_base	导航栈：全局规划 + 局部避障 + 控制
DWA	move_base 的局部规划器
costmap	代价地图，给障碍物周围做膨胀
TF	坐标系变换链，串起 map/odom/base/传感器
/cmd_vel	底盘速度指令话题
看门狗	真车固件里"长时间没收到指令就自动停"的安全机制
IoU	交并比，衡量两张地图/区域的相似度
RMSE	均方根误差，衡量轨迹估计的偏差
rosbag	录制/回放话题数据的文件，用于可复现实验

B 计划（做完 A 之后）

把上面每一节**合上文档，用自己的话讲一遍**（不用术语、像跟同学聊天），讲给我听，我会：

1. 指出你讲得含糊或讲错的地方；
2. 告诉你哪几句可以直接用作复试话术；
3. 最后挑 3 个最可能被追问的问题模拟面试。

建议顺序：先讲"总纲 3 分钟版"，再按博客 01 $\rightarrow$ 博客 02 逐节过。